

Three Astronomy Signal Puzzles for 7th Grade

Computational Thinking with Divisibility, Primes, and Modular Arithmetic

Puzzle 1: The Voyager Signal Code

Story Setup

In 1977, NASA launched Voyager 1 with a Golden Record containing sounds and images from Earth. But engineers also programmed a secret “beacon signal ID” that would help identify the spacecraft if it’s ever found by another civilization—or if we lose track of it ourselves.

The engineers designed the ID using **divisibility rules** so that any mathematician (human or alien) could verify it’s authentic. The ID is a number, and it must follow a specific set of mathematical rules. Your job: **decode the correct signal ID from the transmission logs.**

The Rules (Example: 5-digit Signal ID)

The Voyager beacon ID is a **5-digit number** where:

1. The entire number is **divisible by 3** (sum of digits divisible by 3)
2. The **last 2 digits** form a number **divisible by 4**
3. The **last digit** is **even**
4. The **first 3 digits** form a number that is **divisible by 7**
5. The **middle digit** (position 3) is **prime**
6. The sum of the **first two digits** equals the **last digit**
7. All digits are **different**

Small-Scale Example (3-digit version for hand solving)

Find a **3-digit number** where:

1. The entire number is **divisible by 3**
2. The **last digit** is **even**
3. The **first digit** is **prime**
4. The **second digit** equals the sum of the first and last digits (mod 10)

Solution: Let’s try: First digit = 2 (prime), last digit = 4 (even)

- Second digit = $(2 + 4) \bmod 10 = 6$
- Number: 264
- Check: $2+6+4 = 12$, divisible by 3 ✓
- Last digit even ✓
- First digit prime ✓
- Second = first + last ✓

Answer: 264

Python Solver (5-digit version)

```
def check_voyager_signal(n):
    s = str(n)
    if len(s) != 5 or len(set(s)) != 5: # All different
        return False
    digits = [int(d) for d in s]

    # Rule 1: Divisible by 3
    if sum(digits) % 3 != 0:
        return False
    # Rule 2: Last 2 digits divisible by 4
    if int(s[3:5]) % 4 != 0:
        return False
    # Rule 3: Last digit even
    if digits[4] % 2 != 0:
        return False
    # Rule 4: First 3 digits divisible by 7
    if int(s[0:3]) % 7 != 0:
        return False
    # Rule 5: Middle digit is prime
    if digits[2] not in [2, 3, 5, 7]:
        return False
    # Rule 6: First two sum to last
    if (digits[0] + digits[1]) % 10 != digits[4]:
        return False

    return True

for num in range(10000, 100000):
    if check_voyager_signal(num):
        print(f"Voyager Signal ID: {num}")
```

Puzzle 2: The Exoplanet Transit Timer

Story Setup

Astronomers discovered a distant star system with multiple planets. When a planet passes in front of its star (a “transit”), the star’s brightness dips for a short time. Each planet has a unique **orbital period** (how many days it takes to orbit once).

The planets have synchronized in a rare pattern—they all transited on **Day 0**. NASA wants to know: **On which future day will ALL planets transit together again?**

But there’s a twist: The telescope can only observe for a limited window, and you need to find the **first day** (smallest number) when all planets align.

The Rules (Example: 4-planet system)

Planet orbital periods (in Earth days):

- Planet A: 6 days
- Planet B: 8 days
- Planet C: 9 days
- Planet D: 12 days

Question: What is the **next day** (after Day 0) when all four planets transit simultaneously?

Answer: Find the **Least Common Multiple (LCM)** of 6, 8, 9, 12

Small-Scale Example (3-planet version for hand solving)

Planets:

- Planet X: 4 days
- Planet Y: 6 days
- Planet Z: 10 days

Hand solution:

- Planet X transits on days: 0, 4, 8, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 52, 56, 60...
- Planet Y transits on days: 0, 6, 12, 18, 24, 30, 36, 42, 48, 54, 60...
- Planet Z transits on days: 0, 10, 20, 30, 40, 50, 60...

First alignment: Day **60**

Or use LCM: $\text{LCM}(4, 6, 10) = 60$

Python Solver

```
import math

def lcm(a, b):
    return abs(a * b) // math.gcd(a, b)

def find_alignment(periods):
    result = periods[0]
    for period in periods[1:]:
        result = lcm(result, period)
    return result

# Example: 4-planet system
planets = [6, 8, 9, 12]
alignment_day = find_alignment(planets)
print(f"All planets align on Day {alignment_day}")

# Bonus: Show first 5 alignments
for i in range(1, 6):
    print(f"Alignment #{i}: Day {alignment_day * i}")
```

Computational thinking insight: Students can solve small cases by listing multiples, but with 6+ planets or large periods (e.g., 47, 83, 101), the computer becomes essential.

Puzzle 3: The Pulsar Frequency Decoder

Story Setup

Pulsars are rotating neutron stars that emit regular pulses of radio waves—like cosmic lighthouses. Astronomers detected a strange pulsar that emits pulses at varying frequencies. The pulses seem to encode a message!

Each pulse frequency is a **number between 100-999** (measured in Hz), and the sequence follows a hidden pattern. Your mission: **Find the next frequency in the sequence** and **decode the pattern**.

The twist? The pattern is based on **modular arithmetic** (clock math) and **prime factorization**.

The Rules (Example Pattern)

Observed sequence:

```
Pulse 1: 147 Hz
Pulse 2: 258 Hz
Pulse 3: 369 Hz
Pulse 4: ??? Hz
```

Pattern rule:

- Each frequency is a 3-digit number
- The sum of digits increases by 3 each time
- Each number is divisible by (its position in sequence $\times 3$)
- The middle digit is always even

Small-Scale Example (hand solving)

Given sequence:

```
Pulse 1: 123
Pulse 2: 246
Pulse 3: ???
```

Pattern: Each number is the previous number $\times 2$

Answer: $246 \times 2 = 492$

But let's use a constraint-based approach:

Given:

- Pulse 1: Sum of digits = 12
- Pulse 2: Sum of digits = 15
- Pattern: Sum increases by 3

Pulse 3 constraints:

- Sum of digits = 18
- Divisible by 9 (position 3×3)
- Middle digit is even

Candidates with digit sum 18:

- 189, 198, 279, 288, 297, 369, 378, 387, 396, 459, 468, 477, 486, 495, etc.

Divisible by 9: All of them (digit sum 18 $\rightarrow$ divisible by 9) ✓

Middle digit even: 189, 279, 288, 369, 468, 486

Additional constraint (for unique solution): First digit = previous middle digit + 1

- Pulse 2 middle digit = 4
- Pulse 3 first digit = 5

Answer: 486

Python Solver

```
def check_pulsar(n, position):
    s = str(n)
    if len(s) != 3:
        return False
    digits = [int(d) for d in s]

    # Rule: Divisible by (position × 3)
    if n % (position * 3) != 0:
        return False
    # Rule: Middle digit even
    if digits[1] % 2 != 0:
        return False
    # Rule: Sum of digits = 12 + (position-1) × 3
    target_sum = 12 + (position - 1) * 3
    if sum(digits) != target_sum:
        return False

    return True

def find_next_pulse(sequence):
    position = len(sequence) + 1
    candidates = []

    for n in range(100, 1000):
        if check_pulsar(n, position):
            # Additional constraint: first digit = prev middle + 1
            prev_middle = int(str(sequence[-1])[1])
            if int(str(n)[0]) == prev_middle + 1:
                candidates.append(n)

    return candidates

sequence = [123, 246]
next_pulses = find_next_pulse(sequence)
print(f"Possible next frequencies: {next_pulses}")
```

Puzzle Generator (for teachers)

This Python program generates unique Voyager-style puzzles with exactly one solution:

```

import random

def generate_voyager_puzzle():
    """Generate a unique 5-digit puzzle with one solution."""
    # Start with a random valid signal
    while True:
        digits = random.sample(range(10), 5)
        n = int(''.join(map(str, digits)))

        if check_voyager_signal(n):
            print(f"✓ Generated puzzle with solution: {n}")
            print("\nRules:")
            print("1. All digits are different")
            print("2. Divisible by 3")
            print("3. Last 2 digits divisible by 4")
            print("4. Last digit even")
            print("5. First 3 digits divisible by 7")
            print("6. Middle digit is prime")
            print("7. First two digits sum to last digit (mod 10)")
            return n

# Generate 3 unique puzzles
for i in range(3):
    print(f"\n{'='*50}")
    print(f"PUZZLE #{i+1}")
    print(f"{'='*50}")
    generate_voyager_puzzle()

```

Teaching Timeline (90-minute class)

Phase 1: Exploration (40-45 minutes)

1. **Story introduction** (5 min) - Present the astronomy scenario
2. **Hand-solve small version** (15 min) - Students work in pairs on 3-digit or 3-planet version
3. **Share approaches** (10 min) - Discuss strategies on the board
4. **Scale challenge** (10 min) - Try 5-digit/6-planet version by hand—realize it's tedious!

Phase 2: Computational Thinking (10 minutes)

1. **Decompose** - Break into individual rule checks
2. **Pattern recognition** - Notice rules are independent tests
3. **Abstraction** - "Check if number satisfies all conditions"
4. **Algorithm design** - Sketch pseudocode on board

Phase 3: Python Implementation (35-40 minutes)

1. **Minimal syntax intro** (10 min) - `for`, `if`, `%`, string slicing, lists
 2. **Build solver together** (15 min) - Type code line-by-line
 3. **Run and verify** (5 min) - See computer solve instantly!
 4. **Modify and experiment** (10 min) - Change rules, see what happens
-

Why These Work

- ✓ **Story-driven** - Real astronomy context
- ✓ **Hand-solvable at small scale** - 3-digit numbers, 3 planets
- ✓ **Demonstrates computer power** - Instant solving of large cases
- ✓ **Computational thinking** - Decomposition, pattern recognition, algorithm design
- ✓ **Minimal Python** - ~15-20 lines, chalkboard-friendly
- ✓ **Elegant solutions** - Clean logic, satisfying “aha!” moments
- ✓ **Math connections** - Divisibility, LCM/GCD, modular arithmetic, primes
- ✓ **Exactly one answer** - Or controllable multiple solutions for discussion

Each puzzle teaches different CS concepts:

- **Puzzle 1** - Constraint satisfaction, filtering
- **Puzzle 2** - Number theory (LCM), recursive algorithms
- **Puzzle 3** - Sequence patterns, modular arithmetic